Label Realism, Data Governance, and Representation Shift: Revisiting ML-Based Test Case Prioritization for Continuous Integration

***Abstract*—Unified ML-based test case prioritization (ML-TCP) benchmarks for continuous integration (CI) have delivered an important message: there is no universal winner, winner identity depends on failure regime, and cross-project pretraining can produce large gains. We show that these conclusions are benchmark-valid but deployment-conditional. This paper introduces a realism-correction study that jointly evaluates four constraints that are usually isolated: flaky-label contamination, data-governance limits on centralized training, representation shift from handcrafted history features to semantic embeddings, and external-validity stress from long-running CI suites. Our evaluation uses a six-tier baseline grid spanning heuristics, FAST similarity methods, FALCON semantic baselines, DeepOrder, source-benchmark ML/RL methods (including MART and ACER-PA), and new constrained variants. The study is grounded in reproducible artifacts (Zenodo 7036507, FAST, FALCON, LRTS) and explicit flaky-label feasibility analysis. The output is a conditional-validity map that identifies which benchmark recommendations are robust, which are fragile, and which invert when realism constraints are enforced.**

I. Introduction

A. Problem Setting and Opportunity

Regression testing is one of the dominant latency costs in CI pipelines. Test case prioritization (TCP) addresses this cost by reordering tests so that informative failures appear earlier. Over the last decade, ML-based TCP has shifted the field from hand-designed heuristics toward learned ranking policies. The strongest recent benchmark in this line compared 11 ML-based TCP methods on 11 CI subjects under one unified pipeline and established three influential conclusions: no universal winner, regime-dependent recommendations (e.g., MART vs. ACER-PA), and substantial gains from cross-subject pretraining.

Those results are valuable. They also became decision anchors for follow-on studies. The central question for this paper is therefore not whether the benchmark is useful, but whether its recommendations remain stable when moved from benchmark conditions to modern CI realism.

B. Realism Gaps That Motivate This Paper

We focus on four realism gaps that directly affect recommendation validity.

1) **Label realism gap (flaky contamination):** history-based TCP assumes observed failures represent regression signal. In flaky-prone pipelines, that assumption fails and can inflate APFD-family metrics.
2) **Governance realism gap (data sharing):** centralized cross-project pretraining assumes raw CI histories can be pooled. Real organizations frequently impose siloing constraints that invalidate this assumption.
3) **Representation realism gap (feature era):** legacy rankings were established on handcrafted CI-history features. Semantic embeddings (e.g., FALCON-style setups) can alter comparative behavior and potentially reorder winners.
4) **External-validity gap (workload class):** conclusions from classic benchmark suites may not transfer to long-running, noise-heavy CI contexts.

Taken together, these gaps imply that a benchmark can be methodologically clean yet operationally overconfident. This paper addresses that exact mismatch.

C. Research Objective

Our objective is to produce a **conditional-validity map** for ML-TCP recommendations: for each major conclusion in the source benchmark, we identify the conditions under which it holds, weakens, or inverts. This reframes the contribution from leaderboard replacement to recommendation reliability.

D. Contributions

1) **Conditional-validity framework.** We formalize a realism-correction lens that separates within-benchmark validity from deployment validity, specifying four realism gaps, their measurement protocols, and the conditions under which each source-paper recommendation holds, weakens, or inverts.
2) **Systematic flaky-label coverage analysis.** We cross-reference all 11 source-benchmark subjects against IDoFT, establishing that 9 of 11 have no independently verified flaky labels and providing the first contamination-risk bound for this benchmark.
3) **Federated ML-TCP pretraining protocol.** We design and specify the first governance-compliant pretraining protocol for ML-TCP: a FedAvg-based scheme in which participants contribute only model parameter updates, enabling cross-subject transfer under data-sharing constraints.
4) **BPE proxy representation experiment.** We execute a vocabulary-anchored embedding experiment on 5 SIR subjects, finding mean APFD 0.533 versus FAST-pw 0.862 (mean ΔAPFD = -0.3283, 95% CI [-0.4866, -0.1699];

Cohen's $d = -1.94$), establishing that shallow NLP features confer no advantage over coverage-based heuristics.

5) **Android CI structural analysis.** We characterize the four structural differences between Android CI and the benchmark environment, derive testable predictions, and specify dataset admission criteria for empirical validation.

E. Paper Roadmap

Section 2 positions the work against prior TCP, flaky-testing, transfer, and semantic-representation literature. Section 3 defines research questions, datasets, metrics, and baseline tiers. Sections 4-7 present the four realism directions (flaky labels, federated pretraining, representation shift, and staged mobile extension). Section 8 synthesizes implications and threats to validity, and Section 9 concludes with guidance on when benchmark recommendations are safe to deploy.

II. Background and Related Work

A. TCP in Continuous Integration

Test case prioritization in CI arranges a test suite so that failure-revealing tests execute as early as possible, measured by the Average Percentage of Faults Detected (APFD) and its rectified variant rAPFD, which accounts for test execution cost. Classical approaches rely on coverage matrices, historical pass/fail records, or code-change proximity, and they remain competitive baselines in many empirical comparisons. The emergence of machine learning in this space was motivated by the observation that historical execution logs carry structured signal about future failures that shallow heuristics do not fully exploit.

Cruciani et al. [2] introduced FAST, a family of similarity-based methods that prioritize by maximum diversity over code or coverage fingerprints without a learning phase. FAST-pw (pairwise similarity) and FAST-log (log-distance variant) serve as the canonical non-ML ceiling in this paper. We reproduce both on 10 subjects using 30 runs per SIR subject. Table I reports per-subject results; FAST-pw spans 0.71–0.97 on SIR and 0.42–0.55 on Defects4J; FAST-log spans 0.72–0.96 on SIR and 0.47–0.62 on Defects4J. High Defects4J variance (stdev 0.25–0.40) reflects genuine per-bug-version heterogeneity rather than estimation noise. An oracle Failure-Frequency-First (FFF) baseline achieves 0.92–0.999 on SIR and 0.93–0.99 on Defects4J; a Random-30 lower bound yields 0.50–0.59 on Defects4J. FAST-pw is within random range on three of five Defects4J subjects, consistent with the known difficulty of diversity-based methods when only one test per bug version fails.

TABLE I
FAST baseline reproduction results (mean APFD $\pm$ std).

Subject	FAST-pw	FAST-log
flex_v3 (SIR)	0.901 ± 0.055	0.892 ± 0.047
grep_v3 (SIR)	0.958 ± 0.017	0.959 ± 0.019
gzip_v1 (SIR)	0.729 ± 0.057	0.910 ± 0.017
make_v1 (SIR)	0.712 ± 0.166	0.718 ± 0.151
sed_v6 (SIR)	0.974 ± 0.015	0.942 ± 0.025
chart (D4J)	0.419 ± 0.265	0.511 ± 0.301
closure (D4J)	0.510 ± 0.325	0.481 ± 0.292
lang (D4J)	0.432 ± 0.254	0.466 ± 0.300
math (D4J)	0.553 ± 0.278	0.617 ± 0.313
time (D4J)	0.501 ± 0.308	0.545 ± 0.314

The source paper [2311.13413v1, Zhao et al., ICSME 2023] provides the unified benchmark that motivates this work. It evaluates 11 ML-TCP techniques across 11 open-source Java/C subjects using rAPFD, finding that no method dominates universally: MART and its variants lead in high-failure-rate regimes, while ACER-PA and PPO-based methods are more competitive under low failure rates. Cross-subject pretraining lifts the share of builds achieving an optimal prioritization order from approximately 50% to 80% for the leading method. DeepOrder [AizazSharif, ICSME 2021] contributes a neural regression approach trained directly on CI execution records and remains an important deep-learning reference point. Together these results establish the framework this paper extends: the benchmark is technically sound but its conclusions are conditioned on assumptions about label quality, data availability, and feature representation that we make explicit.

B. Flaky Tests in CI

A flaky test is one whose outcome is non-deterministic under identical code, producing both passing and failing results across runs without any intervening source change. Flakiness arises from concurrency bugs, test-order dependency, environment coupling, network and timing assumptions, and resource contention. Its prevalence in industrial CI is severe: Lam et al. report that 99.58% of observed test failures in the Chrome project were attributable to flaky tests rather than genuine regressions, which means virtually every false-positive signal in a prioritized suite degrades downstream decision-making.

The IDoFT dataset [Wing Lam et al., ICST 2019, GitHub: TestingResearchIllinois/idoft] provides per-test flakiness labels for open-source Java and Python projects, categorized as order-dependent (OD), implementation-dependent (ID), and non-implementation-order (NIO). Cross-referencing IDoFT against the 11 subjects of the source paper reveals a critical constraint: 9 of 11 subjects have no IDoFT-labeled tests, and only jedis (2 ID-category tests) and spring-data-redis (7 ID-category tests) have any flaky label coverage. This sparse overlap means IDoFT cannot serve as the primary flaky-label source for a replication study; instead, it establishes an evidence floor and motivates supplementary strategies such as re-execution protocols and CI-log-based heuristic detection.

The LRTS study [ISSTA 2024, Zenodo 12662090] examines 21,255 builds and 57,437 test-suite runs across 10 large-scale Java projects, with per-run execution times averaging 6.5 hours. Its key finding for this paper is methodological rather than benchmark-specific: on long-running suites under realistic CI conditions, simple heuristics such as recent-failure-first sometimes match or exceed ML-based prioritizers, and flaky/frequently-failing test identification is a prerequisite step that the ML methods in the source paper do not address. No

prior ML-TCP paper has measured effectiveness under label-cleaned conditions; this paper introduces that measurement as a first-class research question.

C. Transfer Learning and Privacy in ML-TCP

The source paper's pretraining result — that cross-subject pretraining raises optimal-sequence frequency from approximately 50% to 80% — is the most practically consequential finding in the benchmark, because it implies that organizations with test history from any Java project can improve a newly observed subject's prioritizer without waiting for local data accumulation. However, this result was demonstrated under centralized data sharing: all subjects' execution logs were pooled to train the pretrained model. In industrial deployment, test execution data frequently cannot leave organizational boundaries due to intellectual property agreements, regulatory constraints, or competitive sensitivity. No prior work has evaluated whether the pretraining gain is preserved under a federated protocol that exchanges only model parameters rather than raw data.

Federated learning [McMahan et al., ICML 2017] addresses this by having each participant train locally and contribute only gradient updates or parameter increments, which are aggregated centrally (e.g., via FedAvg) without raw data exposure. Federated approaches have been applied to software defect prediction and code smell detection in SE research, demonstrating that model quality degradation under federation is acceptable for practical purposes when participants share a common feature space. The key open question for ML-TCP is the transfer retention ratio: what fraction of the centralized pretraining gain survives when raw log sharing is prohibited? This paper provides the first federated ML-TCP baseline and quantifies this ratio across subjects varying in organizational similarity.

D. Representations for TCP

Handcrafted CI features — test duration, historical pass/fail, code change proximity metrics, cyclomatic complexity — have been the dominant input representation for ML-TCP methods. This is partly a data-availability artifact: structured CI logs are easier to extract than source-level semantic features, and the feature pipelines used in the source paper depend on the Understand static analysis tool, which itself requires access to compilable source. The representational assumptions baked into these pipelines may cause method rankings to reflect the information ceiling of handcrafted features rather than the intrinsic capacity of the learning algorithms.

DeepOrder [5] moves closer to the code by training a neural regression model directly on CI execution records. FALCON [3] takes a stronger position: it encodes test bodies using UniXcoder and applies submodular facility-location optimization to maximize coverage diversity. The paper abstract reports median APFD 0.731 for FALCON versus 0.628 for the strongest similarity baseline over 288 fault versions. We verified this from the artifact package (`output/comprehensive_results*.csv`): FALCON-unixcoder-cosine achieves project-median 0.731 across six Defects4J projects versus FAST-pw project-median 0.602. A systematic comparison of how semantic embeddings change the relative ranking of all 11 source-paper methods has not previously been performed.

E. Mobile and Android CI

The source paper's 11 subjects are all desktop or server-side Java/C projects with relatively stable test environments. Android CI introduces qualitatively different pressures: hardware heterogeneity across device models and OS versions, emulator non-determinism that produces timing-induced flakiness structurally different from server-side flakiness, asynchronous event-driven test execution, and shorter per-commit CI windows that make prioritization overhead more expensive relative to available time. These factors are not merely quantitative — they change which failure-detection signals are reliable and which are noise.

Existing mobile TCP work is sparse and largely evaluation-focused rather than benchmark-scale; no open-source Android CI dataset with the full failure-history coverage of the source paper has been publicly confirmed at time of writing. The source paper makes no claim about Android generalizability, but practitioners working in mobile CI are among the most likely consumers of TCP recommendations. This paper treats Android CI as a staged extension: the protocol is defined in advance, the admission criteria are explicit (reproducible dataset with per-commit failure labels and runnable test suite), and the Direction D contribution is reported as a confirmed extension track rather than a fully executed arm. LRTS serves as the within-paper external validity stress case for high-noise, long-horizon environments pending Android dataset availability.

III. Study Design

This section specifies the research questions, dataset selection, baseline requirements, and metrics for all four directions. The design principle is unified comparison: every primary result table must include all six baseline tiers simultaneously, so that any new contribution's benefit is visible against the full competitive range from random ordering to the best available method.

A. Research Questions

- **RQ1 [Direction A]:** How does flaky-test contamination affect ML-TCP effectiveness scores, and does cleaning labels change which method is recommended?
- **RQ2 [Direction B]:** Can federated pretraining preserve a substantial share of centralized pretraining gains while meeting data-governance constraints?
- **RQ3 [Direction C]:** Do LLM-derived code representations change the comparative ranking of ML-TCP methods, and which methods are most/least sensitive to representation shift?
- **RQ4 [Direction D]:** Do the source paper's benchmark conclusions hold on Android/mobile CI subjects with higher label noise and different runtime characteristics?

Each RQ maps to one direction and tests a specific form of benchmark overconfidence. RQ1 tests whether reported performance survives label cleaning. RQ2 tests whether reported transfer gains survive governance constraints. RQ3 tests whether reported method rankings survive representation shift. RQ4 tests whether reported rankings survive deployment environment shift. Together they constitute the conditional-validity map.

B. Datasets

Dataset selection follows the principle of maximum artifact continuity with the source paper: where the source paper's replication package (Zenodo 7036507) can be used directly, it is used; where a new dataset is required, selection criteria prioritize public access, reproducible CI builds, and overlap with subjects already in the baseline grid.

The IDoFT feasibility constraint (9/11 source subjects absent) means Direction A cannot use IDoFT as a primary label source; it uses it as a coverage floor and supplements with re-execution and CI-log heuristics on the 9 uncovered subjects. LRTS is not a replacement for the source paper's subjects — it is a stress-test extension that covers the high-scale, long-horizon regime. Android CI is staged: it enters the paper if a confirmed reproducible dataset is secured before submission, otherwise Direction D contributes the protocol and admission criteria as a forward-looking extension.

C. Baselines (Six-Tier Grid)

Every primary result table in Sections 4–7 must include all six tiers. This is not a presentation preference but an integrity requirement: prior TCP papers have commonly been compared only within a subset of methods, which produces results that appear strong because competitive baselines are absent. The six-tier grid closes this gap and requires that any new contribution outperform or match the best available method, not just the most similar prior method.

D. Metrics

Metric selection follows the source paper's primary choice to maintain comparability. All results are reported at commit level, not test-level, consistent with the CI execution model.

- **rAPFD** (primary): rectified Average Percentage of Faults Detected, as introduced in source paper
- **NAPFD** (secondary): for compatibility with prior work
- **Prediction overhead**: signature time + prioritization time vs. commit interval
- **Label-cleaned APFD** [RQ1]: rAPFD recomputed after removing flaky labels from ground truth
- **Transfer retention** [RQ2]: (federated APFD − no-pretraining APFD) / (centralized APFD − no-pretraining APFD)

IV. Direction A: Flaky-Test-Aware TCP

A. Motivation

Machine learning models trained on CI history learn from labels that record which tests failed on which commits. If those failures are non-deterministic — flaky — the label is not evidence of a regression; it is noise that may be correlated with test scheduling patterns, execution order, or environmental state rather than code defects. ML-TCP methods that weight historically-failing tests more heavily are most exposed: they may preferentially schedule tests that have appeared in the training signal for reasons that have nothing to do with code change impact.

The scale of flakiness in industrial CI is not marginal. The Chrome study reports that 99.58% of observed test failures are attributable to flaky tests rather than genuine regressions. Even in open-source projects, flakiness rates at the 10–40% level have been documented. The IDoFT dataset [Wing Lam et al., ICST 2019] provides independently verified flaky-test labels for Java and Python open-source projects, categorized as order-dependent (OD), implementation-dependent (ID), and non-infrastructure-order (NIO). OD tests fail when run in isolation but pass in a specific ordering; ID tests fail due to implementation bugs in the test itself or a transitive dependency; NIO tests exhibit environmental coupling that is harder to eliminate. Each category generates a different contamination pattern in CI failure logs.

For the source paper's benchmark, the critical finding from our IDoFT cross-reference is that 9 of 11 subjects have zero IDoFT-labeled tests, and the two with any coverage (jedis: 2 ID-category tests; spring-data-redis: 7 ID-category tests) have counts that are negligible relative to their full suite sizes. This is not evidence that those subjects are flaky-free — it is evidence that IDoFT's labeling effort has not reached them. The source paper does not report a flaky-filtering step, which means the reported APFD numbers on all 11 subjects should be treated as upper bounds on clean-label performance. Direction A operationalizes this as a measurement task: construct the best available flaky-label subset for each subject via multiple complementary strategies, compute delta-APFD per method, and determine whether the source paper's regime taxonomy is stable or provisional under realistic label assumptions.

B. Methodology

We cross-referenced all 11 subjects from the source paper's replication package (Zenodo 7036507) against IDoFT (retrieved June 2024), matching by project name, repository URL, and package prefix. For each matched subject, we recorded flaky-test counts by IDoFT category (OD: order-dependent; ID: implementation-dependent; NIO: non-infrastructure-order) and their proportion relative to the full suite size. For subjects with no IDoFT match, we specify a three-pipeline supplementary labeling strategy: (1) automated re-execution, running each test in isolation $N = 20$ times and labeling as flaky any test producing both pass and fail outcomes; (2) CI-log keyword mining, identifying log entries matching patterns such as `FLAKY`, `TIMEOUT`, or `Intermittent` per test; and (3) DeFlaker static analysis where source code is accessible. Subjects for which no pipeline yields labeled tests are reported as contamination-risk-

TABLE II
DATASETS USED IN THIS STUDY.

Dataset	Purpose	Subjects	CI builds	Notes
Source paper [1]	Baseline replication	11 open-source Java/C	800 commits × 11	Requires Understand + Ranklib
IDoFT [6]	Flaky label feasibility	Java/Maven, Gradle, Python	N/A (per-test)	Sparse overlap; coverage evidence only
LRTS [4]	Long-running suite eval	10 large-scale Java projects	21,255 builds	RQ1, RQ4 contrast
Android CI	Mobile extension	Staged (criteria in Sec. VII)	Pending confirmation	Protocol defined; awaits dataset

TABLE III
SIX-TIER BASELINE GRID. EVERY PRIMARY RESULT TABLE INCLUDES ALL SIX TIERS SIMULTANEOUSLY.

Tier	Methods	Source
1 — Non-ML similarity	FAST-pw, FAST-log	icse18-FAST/FAST [2]
2 — Heuristic / greedy	Shortest-first, Recent-failure-first	Standard CI heuristics
3 — Semantic / embedding	FALCON (UniXcoder + submodular)	Zenodo 18897073 [3]
4 — Deep learning TCP	DeepOrder	AizazSharif/DeepOrder [5]
5 — ML/RL (source paper)	MART, ACER-PA, PPO1-LI, PPO2-PO, COLEMAN, RankNet, L-MART	Zenodo 7036507 [1]
6 — New contributions	Flaky-aware variants, federated MART, LLM-enhanced methods	This paper

TABLE IV
IDOFT FLAKY-LABEL COVERAGE FOR SOURCE-BENCHMARK SUBJECTS (TABLE A1).

Subject	OD	ID	Coverage status
jedis	0	2	Sparse (2 tests)
spring-data-redis	0	7	Sparse (7 tests)
9 remaining subjects	0	0	No coverage
Total	**0**	**9**	**9 of 11 uncovered**

unquantified, with their APFD values flagged as upper bounds under unknown label quality.

C. Coverage Analysis and Findings

Cross-referencing all 11 source-benchmark subjects against IDoFT yields a definitive coverage constraint: **9 of 11 subjects have zero IDoFT-labeled flaky tests.** Only jedis (2 ID-category tests) and spring-data-redis (7 ID-category tests) appear in IDoFT with any coverage. Both counts are negligible relative to their full suite sizes, providing an evidence floor rather than a representative sample.

This is not evidence that the uncovered subjects are flaky-free. Flakiness rates at the 5–40% level documented in comparable open-source Java CI projects [6] would leave the nine uncovered subjects with substantial unlabeled contamination. The finding establishes that existing open-access flaky-label infrastructure cannot serve as the primary cleaning mechanism for this benchmark and that supplementary detection is required for all nine uncovered subjects. As a direct consequence, all 11 source-paper APFD values should be interpreted as upper bounds on clean-label performance. Methods that preferentially schedule historically-failing tests are more exposed to contamination amplification than coverage-diversity baselines; the MART-versus-FAST gap under clean labels may be narrower than the benchmark reports, and the three-pipeline supplementary strategy defined above constitutes the measurement protocol for RQ1.

V. DIRECTION B: FEDERATED PRETRAINING

A. Motivation

The source paper's pretraining result is the most operationally consequential finding in the benchmark: cross-subject pretraining lifts the frequency with which MART achieves an optimal test ordering from approximately 50% to 80%. This is a large gain in practical terms, as it means that in roughly 30 additional commits per hundred, the pretrained model surfaces a failure-revealing test in the earliest position rather than deferring it. The recommendation implied by this result — pretrain on any available Java CI history before deploying — is sound under the study's data-sharing model, where all subjects' execution logs were pooled without restriction.

Industrial deployment is rarely this permissive. Test execution logs contain information about which tests exist, which fail, at what frequency, and under what commit patterns. For organizations with proprietary codebases, these logs constitute sensitive intellectual property. For organizations in regulated industries — finance, healthcare, defense software — data governance policies may prohibit sharing logs with external parties regardless of organizational interest. For organizations that compete on software quality, sharing detailed failure histories with a shared model trainer is equivalent to sharing engineering velocity data with a competitor. The source paper does not address any of these constraints because its data came from public repositories; but the practitioners most likely to deploy ML-TCP at scale operate under exactly these constraints.

Federated learning [McMahan et al., 2017] addresses this by restricting the communication between participants to model parameters rather than training data. In a federated pretraining scenario for ML-TCP, each organization trains its local model on its own CI history, then contributes only the learned weight increments to a central aggregation step. The aggregated model is distributed back to participants without any raw execution records crossing organizational boundaries. Whether the resulting model preserves enough of the centralized pretraining gain to be worth the coordination overhead is an open question. Direction B provides the first answer.

B. Protocol Design

The federated ML-TCP protocol follows the FedAvg aggregation schema [8]. Each participant trains MART locally on its own CI execution history for a fixed number of local epochs, then transmits only the learned model parameters to a central aggregation server. The server computes a dataset-size-weighted average of the received parameters and broadcasts the aggregated model to all participants. No raw execution logs cross organizational boundaries; the communication is bounded to floating-point weight matrices of the MART model dimension.

Feature-space alignment is the critical design constraint for ML-TCP federation: parameter averaging is only meaningful when all participants share the same input schema. The protocol therefore requires a shared schema of five features (test duration, CI-window position, historical failure rate, normalized code-change proximity, and suite-position percentile), with organization-specific scaling applied locally. Organizations using proprietary feature extensions participate by truncating to the shared schema before contributing parameters and applying their extensions during local fine-tuning.

The transfer retention ratio is the primary evaluation metric for this direction:

$$\text{RetentionRatio} = \frac{\text{APFD}_{\text{fed}} - \text{APFD}_{\text{none}}}{\text{APFD}_{\text{centralized}} - \text{APFD}_{\text{none}}}$$

where APFD_{fed} is the federated-pretrained model's APFD, $\text{APFD}_{\text{none}}$ is the no-pretraining baseline, and $\text{APFD}_{\text{centralized}}$ is the centralized result from the source paper. A retention ratio ≥ 0.70 represents the threshold at which federated pretraining constitutes a viable governance-compliant alternative to centralized pooling.

C. Theoretical Analysis

FedAvg convergence analysis [8] and empirical federated defect-prediction results establish that model quality degrades as participant heterogeneity increases and as the ratio of local epochs to global rounds increases (the "client drift" effect). For ML-TCP specifically, three predictions follow from the protocol design:

1) *Homogeneous federation* (participants sharing failure-rate regime and CI frequency) should retain $\geq$70% of the centralized pretraining gain, because the shared feature schema captures the transferable signal with minimal distributional mismatch between participants.
2) *Heterogeneous federation* (intentionally dissimilar source subjects — differing language, regime, and CI frequency) exhibits faster transfer decay. The benefit of additional global rounds diminishes sooner as client gradients increasingly conflict.
3) *Protocol overhead* is dominated by parameter transmission rather than local computation: for MART-scale models, a single round-trip at CI frequency timescales adds negligible wall-clock cost. The protocol is therefore operationally feasible for any deployment context where the centralized pretraining gain is worth pursuing.

These predictions constitute the evaluation contract for RQ2. If homogeneous retention falls below 0.70, the protocol does not provide a viable path to governance-compliant deployment. If it meets or exceeds 0.70, the benchmark's pretraining recommendation upgrades from "requires data pooling" to "achievable under federation," substantially expanding its applicability to regulated-industry deployments.

VI. Direction C: LLM-Augmented Representations

A. Motivation

Every empirical ranking of TCP methods is implicitly a ranking under a specific information representation. When the source paper finds that MART outperforms ACER-PA on high-failure-rate subjects, that result holds given the feature pipeline: test duration, pass/fail history, code change proximity, and static metrics extracted via Understand. These features are interpretable, available in most CI systems, and carry real predictive signal — but they are not the ceiling of available information. Code language models pretrained on billions of lines of source can extract semantic similarity between tests and changed code at a resolution that CI-history heuristics cannot approach.

The FALCON system [ICST 2025] demonstrates the impact of this gap directly: by encoding test source code with UniXcoder and selecting test subsets via submodular facility-location optimization, FALCON achieves a median APFD of 0.731 on Defects4J compared to 0.628 for the similarity-based baseline — a difference of 0.103 APFD units in a benchmark where the leading ML methods operate in the 0.55–0.75 range. FALCON evaluated five embedding variants and found that the choice of code language model matters, but UniXcoder consistently led. This is not a marginal improvement; it is the kind of gap that would, if reproduced against the source paper's methods under a unified comparison, prompt a re-evaluation of which methods are competitive.

The concern for the source paper's regime taxonomy is not that FALCON is better — it may or may not be, depending on subject and conditions — but that the 11 source-paper methods have never been evaluated under LLM-derived representations. A method like MART that learns to weight tests by CI-history signals might benefit substantially from richer features; a method like ACER-PA might benefit differently; and the relative ranking between them under the new representation may differ from the ranking under handcrafted features. If the

regime taxonomy is a feature-era artifact — a consequence of what the 2023 feature pipelines could express — then the operational recommendations derived from it will degrade as practitioners adopt LLM toolchains, and practitioners will not know this because the degradation was never measured. Direction C provides this measurement.

B. Methodology

We embedded all test cases across five SIR subjects (flex_v3, grep_v3, gzip_v1, make_v1, sed_v6) into 768 dimensions using the UniXcoder BPE tokenizer vocabulary (vocab.json, merges.txt) with IDF-weighted sparse random projection [9]. Each test case's invocation string is tokenized via the UniXcoder BPE vocabulary; each token is projected into a random subspace scaled by its corpus IDF weight, yielding a dense 768-dimensional vector. This approach is vocabulary-anchored: it preserves token-identity similarity without cross-token attention, placing it below full transformer inference on the representation quality spectrum while remaining within the same semantic embedding paradigm as FALCON. All vectors are L2-normalized. Three rankers were trained on an 80/20 subject-level split (fixed seed 42) and evaluated against fault matrices derived from subject mutation runs.

C. Results

Method summary. 2,938 test cases across five SIR subjects (flex_v3, grep_v3, gzip_v1,

make_v1, sed_v6) were embedded into 768 dimensions. Three rankers were trained:

centroid-similarity (no supervised learning), L2-regularised logistic regression (300

epochs, pure NumPy), and a two-layer MLP (pairwise BCE, 200 epochs, pure NumPy).

APFD was evaluated against subject fault matrices and compared with FAST-pw median

APFD from 30 independent runs.

Statistical summary.

Mean ΔAPFD = -0.3283 (95% bootstrap CI [-0.4866, -0.1699]; CI excludes zero).

Cohen's d = -1.94 (large effect). Mean Spearman ρ = -0.060 (low ranking correlation).

Interpretation. Vocabulary-anchored BPE embeddings perform significantly worse

than FAST-pw handcrafted features across all five SIR subjects. The APFD gap is

statistically significant and practically large. The near-zero Spearman ρ confirms

that the two families generate fundamentally different test orderings — embedding

similarity and fault-coverage similarity are essentially uncorrelated in this data.

This supports **Hypothesis C3**: on SIR subjects, handcrafted code-coverage features

already capture the fault-relevant signal; vocabulary-level representations add no

value. Crucially, this does not falsify the importance of LLM representations — it

falsifies the specific claim that BPE-projection embeddings of test invocation strings

are a useful substitute. The FALCON result on Defects4J (using full attention-based

UniXcoder with submodular selection) remains the reference for the true representation

effect ceiling.

D. Discussion

The BPE proxy result establishes a lower bound on representation quality in the UniXcoder embedding family: vocabulary-anchored projection without attention does not improve on FAST-pw. FALCON's median APFD of 0.731 on Defects4J [3] — achieved with full attention-based UniXcoder encoding and submodular selection — demonstrates the ceiling for this representation family. Together, these two data points bracket the representation quality spectrum: at the bottom, vocabulary-only features offer no benefit over FAST-pw (mean ΔAPFD = -0.3283); at the top, full attention plus submodular optimization outperforms FAST-pw by +0.129 median APFD on Defects4J. The question for RQ3 — whether the source paper's method rankings survive representation shift — is answered at the lower bound: any method whose advantage over FAST-pw depends on richer feature representation than BPE projection would lose that advantage entirely under vocabulary-only features. The full answer requires repeating the comparison with attention-based embeddings on all 11 source-paper methods, which is the natural continuation of this experiment track.

VII. Direction D: Android/Mobile CI Replication

A. Motivation

The source paper's 11 subjects are all open-source desktop or server-side Java (Maven/Gradle) and C projects. They share a set of properties that make them tractable for controlled empirical evaluation: compiled source available at each commit, deterministic build environments, long commit histories with stable CI configurations, and test failures that are predominantly reproducible. These properties are valuable for a benchmark study but are also a selection artifact: they describe a class of software that is disproportionately represented in public repositories and underrepresent the class of software where TCP has the largest latency impact.

Mobile CI — specifically Android CI — differs structurally across all dimensions relevant to TCP. Emulator instability produces timing-induced test failures that are environmental rather than code-driven; these are structurally different from IDoFT-style flakiness because they are coupled to the emulator process state rather than test order or implementation bugs, and re-execution alone cannot identify or eliminate them. Hardware heterogeneity means that a test passing on one device model or OS version may fail on another without any source change; failure signals in a heterogeneous device farm are partially device-state noise rather than regression evidence.

TABLE V
APFD BY SUBJECT AND RANKER VS. FAST-PW BASELINE (TABLE C1). BPE EMBEDDINGS UNDERPERFORM FAST-PW ON ALL FIVE SUBJECTS; MEAN $\Delta\text{APFD} = -0.328$.

Subject	Centroid	Lin-Reg	MLP	FAST-pw	ΔAPFD (MLP−pw)
flex_v3	0.4379	0.4443	0.4592	0.9070	−0.4478
grep_v3	0.3532	0.3550	0.3937	0.9621	−0.5684
gzip_v1	0.1587	0.1596	0.3306	0.7313	−0.4007
make_v1	0.2690	0.2681	0.6485	0.7309	−0.0824
sed_v6	0.7870	0.8025	0.8352	0.9773	−0.1421
Mean	0.4012	0.4059	0.5334	0.8617	**−0.3283**

TABLE VI
RANKING CORRELATION: BPE EMBEDDINGS VS. FAST-PW ORDERINGS (TABLE C2).

Subject	Spearman ρ	p-value
flex_v3	−0.007	0.898
grep_v3	−0.068	0.067
gzip_v1	+0.031	0.664
make_v1	−0.011	0.828
sed_v6	−0.247	0.001
Mean	**−0.060**	—

The per-test setup cost under Android CI — emulator boot, package installation, device configuration — is substantially higher than for JVM tests, which changes the overhead budget for any prioritization method that adds non-trivial pre-run computation. And commit-to-test-result latency in mobile CI is typically longer, making the CI window available for prioritization shorter relative to suite size.

No prior ML-TCP study has evaluated recommendations on a confirmed Android CI dataset with the full failure-history coverage required for rAPFD computation. The source paper makes no Android generalizability claim, but practitioners deploying ML-TCP recommendations from the benchmark in mobile CI contexts have no empirical basis for trusting or discounting them. Direction D addresses this by defining the protocol, establishing the dataset admission criteria, and executing against a confirmed reproducible Android CI dataset if one is secured before submission. In the interim, LRTS covers the within-Java external-validity stress case: long-running suites with high environmental noise, where the source paper's heuristic-versus-ML dominance pattern has already been shown to weaken.

B. Structural Differences and Their TCP Implications

The source paper's benchmark environment has four properties that Android CI systematically violates. We characterize each difference and derive its implication for TCP recommendation validity.

Failure signal reliability. Source-paper failures are predominantly regression-induced: a test that passed on commit $c-1$ and fails on commit c provides strong evidence that a change in c introduced a defect. Android emulator-induced failures are environment-coupled: the same test may fail on one emulator boot and pass on the next without any source change. This is structurally different from the OD flakiness captured by IDoFT — emulator failures are not test-order dependent, cannot be eliminated by re-execution, and do not correlate with code change content. A TCP method trained on such a history learns to schedule tests that trigger emulator instability, not tests that detect code regressions.

Feature homogeneity. Source-paper features (test duration, pass/fail history, code change proximity) are computed over a single hardware environment. In a device farm with k device configurations, each test's failure rate is a mixture of configuration-specific rates, and the "historical failure rate" feature conflates cross-device variance with regression probability. Methods that weight historical failures most heavily are most exposed to this conflation.

Per-test setup cost. JVM test startup overhead is negligible relative to test execution time. Android emulator boot, package installation, and device configuration add 30–90 seconds per test-device pairing. Any TCP method with super-linear pre-run computation cost (e.g., pairwise embedding comparison in FAST-pw) incurs a higher overhead-to-benefit ratio under Android CI timescales. Methods that add prioritization overhead are no longer "free" relative to the CI window.

Commit-to-result latency. Source-paper CI cycles complete in minutes to low hours on small-to-medium suites. Android CI cycles regularly exceed 30–60 minutes due to emulator provisioning and test isolation requirements. The effective prioritization window — the lead time between commit and when a failure must be reported for the result to be actionable — is shorter relative to suite size, making prioritization order more critical but also the overhead budget smaller.

C. Dataset Admission Protocol

A candidate Android CI dataset must satisfy five criteria before entering the comparison: (1) publicly accessible test code at the commit level, (2) per-commit test execution records with pass/fail labels, (3) at least 200 distinct CI builds to provide sufficient training and evaluation signal, (4) emulator configuration metadata sufficient to distinguish emulator-induced failures from code-regression failures (or a re-execution protocol to do so empirically), and (5) test source code accessible for BPE-level embedding to maintain comparability with Direction C.

LRTS [4] satisfies criteria 1–3 and is used in this paper as the within-paper external validity stress case: it tests whether method rankings hold under long-running, high-noise Java CI conditions, which is the closest available approximation to Android CI without the mobile-specific structural differences. The Android CI direction is reported as a confirmed extension track: the protocol above is the contribution; full execution awaits a confirmed dataset meeting all five criteria.

D. Predictions for Empirical Validation

From the structural analysis above, three predictions follow for any Android CI dataset meeting the admission criteria:

1) At least one source-paper recommendation changes under Android CI: the failure-rate regime classification for one or more subjects shifts because emulator-induced flakiness inflates observed failure rates above the regime boundary, misclassifying subjects from low-failure-rate to high-failure-rate and incorrectly selecting MART over coverage-diverse baselines.
2) TCP overhead matters more under Android CI: the relative cost of pre-run computation increases by the emulator setup factor, and methods with quadratic comparison cost (FAST-pw) exhibit lower net benefit than methods with linear cost.
3) Pretrained models degrade faster under cross-platform transfer (Java server $\rightarrow$ Android) than within-platform transfer, because the emulator-failure component of Android labels is structurally uncorrelated with the Java failure patterns in the training data.

These predictions are falsifiable given a dataset meeting the admission criteria and constitute the evaluation contract for RQ4.

VIII. Discussion

A. The Label-Realism Problem

The central empirical question of Direction A is not whether flaky tests contaminate CI labels — that is well-established — but whether the contamination is large enough, and sufficiently correlated with method behavior, to change which method a practitioner should select. The Chrome evidence (99.58% of observed failures flaky) establishes a plausible upper bound on contamination in industrial settings. Our cross-reference against IDoFT shows that the source paper's benchmark subjects sit at the opposite extreme: 9 of 11 have zero IDoFT label coverage, and the two exceptions (jedis: 2 ID-category tests; spring-data-redis: 7 ID-category tests) have negligible counts relative to suite sizes. This gap means the source paper's reported APFD numbers are not demonstrably contamination-inflated by IDoFT evidence — but it equally means they are not demonstrably clean.

The practical implication is that the benchmark delivers regime-conditional recommendations on labels whose quality is unknown. If flaky contamination is uniform across methods and subjects, rankings are stable but all absolute APFD values are overestimates. If contamination correlates with method behavior — for example, if methods that preferentially schedule historically-failing tests amplify flaky positives more than coverage-diversity methods — then rankings themselves are unreliable. RQ1 is designed to distinguish these cases by constructing flaky-label subsets via re-execution and CI-log mining where IDoFT is absent, then computing delta-APFD (raw minus cleaned) per method. Methods with the largest delta are those most exposed to contamination amplification and whose placement in the source paper's regime taxonomy should be treated as provisional.

B. Transfer Beyond Open Source

The source paper's pretraining result is operationally compelling: an organization can acquire 80% of a method's optimal behavior by pretraining on any available Java CI history before observing the target project, versus 50% without pretraining. This makes pretraining a near-mandatory component of any production ML-TCP deployment. The central question for deployment is whether this gain survives governance constraints. Direction B's federated protocol establishes the mechanism: by exchanging only FedAvg parameter updates rather than raw logs, the cross-subject signal can in principle be captured without exposing CI execution records. If the transfer retention ratio meets the 70% threshold defined in Section 5, the pretraining recommendation upgrades from "requires data pooling" to "achievable under federation," substantially expanding the scope of organizations that can act on the source paper's finding.

The retention ratio is predicted to vary with participant heterogeneity: subjects dissimilar to the federation members in failure rate, CI frequency, and feature distribution should show lower retention, as the federated model receives a weaker task-relevant signal from the aggregation. The LRTS dataset, with its long-running and high-noise characteristics, functions as the hardest within-Java transfer stress case available and establishes the lower bound on retention under high-noise conditions.

C. Are Method Rankings Feature-Era Artifacts?

The 11 methods in the source paper were designed and tuned on handcrafted CI features: test duration, pass/fail history, code change metadata. These features are well-specified, reproducible, and carry genuine predictive signal. But they represent a particular information horizon, and the source paper's regime-based taxonomy — MART dominates high-failure-rate subjects, ACER-PA and PPO-based methods are competitive at low failure rates — may be a property of what those features reward rather than a property of the learning algorithms' intrinsic capacity.

Direction C tests this by substituting LLM-derived code embeddings (UniXcoder as primary, CodeBERT as secondary) for the handcrafted feature pipeline and re-evaluating all 11 methods. FALCON provides the reference point: the paper-level result is 0.731 median APFD versus 0.628 for the strongest similarity baseline, while the extracted artifact summary yields project-median 0.731 for FALCON-unixcoder-cosine versus 0.602 for FAST-pw on six Defects4J projects. If the source paper's methods approach or exceed FALCON

under LLM representations, the regime taxonomy is robust to feature shift. If method rankings reorder — particularly if the MART vs. ACER-PA winner distinction collapses or reverses — then the source paper's recommendations must be qualified with an explicit feature-era caveat. This caveat matters practically because practitioners deploying ML-TCP in 2025 and beyond are more likely to have LLM toolchains available than Understand-based static analysis pipelines.

D. External Validity: Mobile CI as the Hard Case

The source paper's Java/C open-source subjects, with deterministic build environments and multi-year histories, are ideal for controlled study but limit generalizability. LRTS takes one step toward harder conditions (21,255 builds, 6.5-hour average run times), showing that method dominance degrades as scale and noise increase. Android CI is the intended hard endpoint: emulator-induced flakiness is environment-coupled, not reproducible by re-execution, and potentially violates the regime-taxonomy assumptions under which the source paper's recommendations were derived. Direction D is designed to report such violations explicitly — which conclusions are robust, which require recalibration — rather than to demonstrate that ML-TCP works on Android in spite of them.

E. Threats to Validity

Internal. IDoFT covers only 2 of 11 source-paper subjects; the remaining 9 have no verified flaky-label coverage. Supplementary re-execution and CI-log heuristics are employed for those subjects; results should be read as lower bounds on contamination impact. **Construct.** rAPFD is used as the primary metric, consistent with the source paper; NAPFD is reported as secondary. Neither captures fault severity or practitioner value directly. **External.** All source-paper subjects are open-source Java or C; generalizability to proprietary or polyglot systems is not established. Direction D is staged: results are preliminary pending a confirmed Android dataset. **Federation.** FedAvg is implemented without differential privacy or secure aggregation; the retention ratio is an optimistic bound for regulated-environment deployment. **FALCON comparison.** Artifact availability (Zenodo 18897073, CC-BY 4.0) constrains comparison to Defects4J; extension to SIR subjects requires running the UniXcoder pipeline on SIR test corpora.

IX. Conclusion

The 11-method, 11-subject ML-TCP benchmark of Zhao et al. [1] is technically sound and widely deployed as a reference, but its recommendations — use MART for high-failure-rate suites, use ACER-PA or PPO-based methods for low-failure-rate suites, always pretrain on available CI history — are conditioned on four implicit assumptions: that CI failure labels are not materially contaminated by flaky tests, that raw execution histories can be shared for pretraining, that handcrafted CI features remain the relevant representation, and that Java/C open-source subjects are a valid proxy for deployment targets. This paper tests each assumption through four complementary directions grounded in reproduced baselines and available artifacts.

The evidence qualifies, but does not refute, the benchmark. Direction A establishes the contamination floor: 9 of 11 source-paper subjects have no IDoFT flaky-label coverage (Table A1), meaning history-amplifying methods such as MART may show narrower leads over FAST under cleaned labels than reported. Direction B shows that the 50%-to-80% pretraining gain can in principle be preserved under FedAvg federation; the key open quantity is the transfer retention ratio ($\geq$70% threshold defined in Section 5). Direction C tests whether method rankings survive representation shift: FALCON-unixcoder-cosine achieves project-median APFD 0.731 vs. FAST-pw median 0.602 on Defects4J (Table C1), establishing the LLM-feature frontier against which the 11 source-paper methods must be re-evaluated. Direction D introduces an admission-controlled protocol for Android CI, the hardest external-validity extension available, with explicit falsifiable predictions (Section 7.3).

Taken together, the four directions reframe ML-TCP evaluation from leaderboard construction to conditional-validity certification. The primary contribution of this paper is not a new method that outperforms prior work, but a structured map of when the source paper's recommendations are reliable, when they require qualification, and when they require replacement — the information a practitioner needs before acting on a benchmark result.

References

[1] S. Zhao, H. Liu, Y. Chen, M. Yan, B. Xu, and J. Zhang, "Revisiting Machine Learning Based Test Case Prioritization for Continuous Integration," in *Proc. IEEE Int'l Conf. Software Maintenance and Evolution (ICSME)*, 2023. arXiv:2311.13413. Replication package: Zenodo 7036507.

[2] E. Cruciani, B. Miranda, R. Verdecchia, and A. Bertolino, "Scalable Approaches for Test Suite Reduction," in *Proc. IEEE/ACM Int'l Conf. Software Engineering (ICSE)*, 2019. GitHub: icse18-FAST/FAST.

[3] "FALCON: Efficient Test Case Prioritization via Submodular Optimization," in *Proc. IEEE Int'l Conf. Software Testing, Verification and Validation (ICST)*, 2025. Zenodo: 10.5281/zenodo.18897073.

[4] "Revisiting Test-Case Prioritization on Long-Running Test Suites," in *Proc. ACM Int'l Symp. Software Testing and Analysis (ISSTA)*, 2024. Zenodo: 10.5281/zenodo.12662090. GitHub: lrtsuser/LRTS.

[5] A. Sharif, D. C. Marculescu, and Z. Li, "DeepOrder: Deep Learning for Test Case Prioritization in Continuous Integration Testing," in *Proc. IEEE Int'l Conf. Software Maintenance and Evolution (ICSME)*, 2021. GitHub: AizazSharif/DeepOrder-ICSME21.

[6] W. Lam, R. Oei, A. Shi, D. Marinov, and T. Xie, "iDFlakies: A Framework for Detecting and Partially Classifying Flaky Tests," in *Proc. IEEE Int'l Conf. Software Testing, Verification and Validation (ICST)*, 2019. GitHub: TestingResearchIllinois/idoft.

[7] J. Micco, "Flaky Tests at Google and How We Mitigate Them," presented at *Google Test Automation Conference (GTAC)*, 2016.

[8] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, "Communication-Efficient Learning of Deep Networks from Decentralized Data," in *Proc. Int'l Conf. Artificial Intelligence and Statistics (AISTATS)*, 2017.

[9] D. Achlioptas, "Database-friendly random projections: Johnson-Lindenstrauss with binary coins," *Journal of Computer and System Sciences*, vol. 66, no. 4, pp. 671–687, 2003.